



Advanced NLP

SPRING 2026

OVERVIEW

1. Embeddings
 - Word2Vec/ Doc2Vec
 - GloVe
 - Exercises
2. Intro to Neural Networks
 - Components, Structure, Learning & Training
3. Intro to PyTorch and Simple Neural Network Colab

Embeddings

Text Vectorization - Embeddings

What are embeddings?

- Representation of words in a dense vector space where similar words are placed closer together.
- Capturing semantic meaning through vector relationships, enabling words with similar contexts to have similar vector representations.
- **Example:** In an embedding space, the vector difference between 'king' and 'queen' is similar to the vector difference between 'man' and 'woman'.

Text Vectorization - Embeddings

Why use embeddings?

- Overcomes limitations of traditional text representations like TF-IDF and bag-of-words models, which ignore word order and context.
- Enables deep learning models to understand context, relationships between words, and even syntactic structures.
- **Example:** In a sentiment analysis task, traditional methods might treat 'not happy' and 'happy' similarly due to word frequencies, while embeddings capture the negation and modify the sentiment meaning.

Text Vectorization - Embeddings

How are embeddings created?

- Training embeddings through unsupervised learning, where models learn to place similar words close to each other based on their co-occurrence in large corpora.
- **Pre-trained embeddings** (like Word2Vec & GloVe) are trained on massive datasets and can be fine-tuned for specific tasks.
- **Learning embeddings from scratch:** Custom embeddings can be trained within deep learning models to optimize for task-specific performance.
- **Example:** In a chatbot system, fine-tuning embeddings on customer support dialogues improves model accuracy compared to using generic embeddings trained on Wikipedia.

Text Vectorization - Embeddings

Example:

Imagine we want to represent a list of animals with numbers rather than words. We could create a list of attributes to describe each animal using numerical values. Let's start with three qualities:

- **Size** (from a numbered list of sizes, i.e. small = 1, medium = 2, large = 3)
- **Habitat** (from a numbered list of habitats, i.e. land = 1, water = 2, air = 3)
- **Legs** (number of legs)

We can represent each animal as a list of three numbers, or a 3-dimensional (3D) vector.

Text Vectorization - Embeddings

Let's define three different animals using this 3-dimensional vector space:

$$[x, y, z] = [\text{size}, \text{habitat}, \text{legs}]$$

lizard: $[1, 1, 4]$ (small, land-dwelling, 4 legs)

snake: $[1, 1, 0]$ (small, land-dwelling, no legs)

spider: $[1, 1, 8]$ (small, land-dwelling, 8 legs)

Difference between Lizard and Snake:

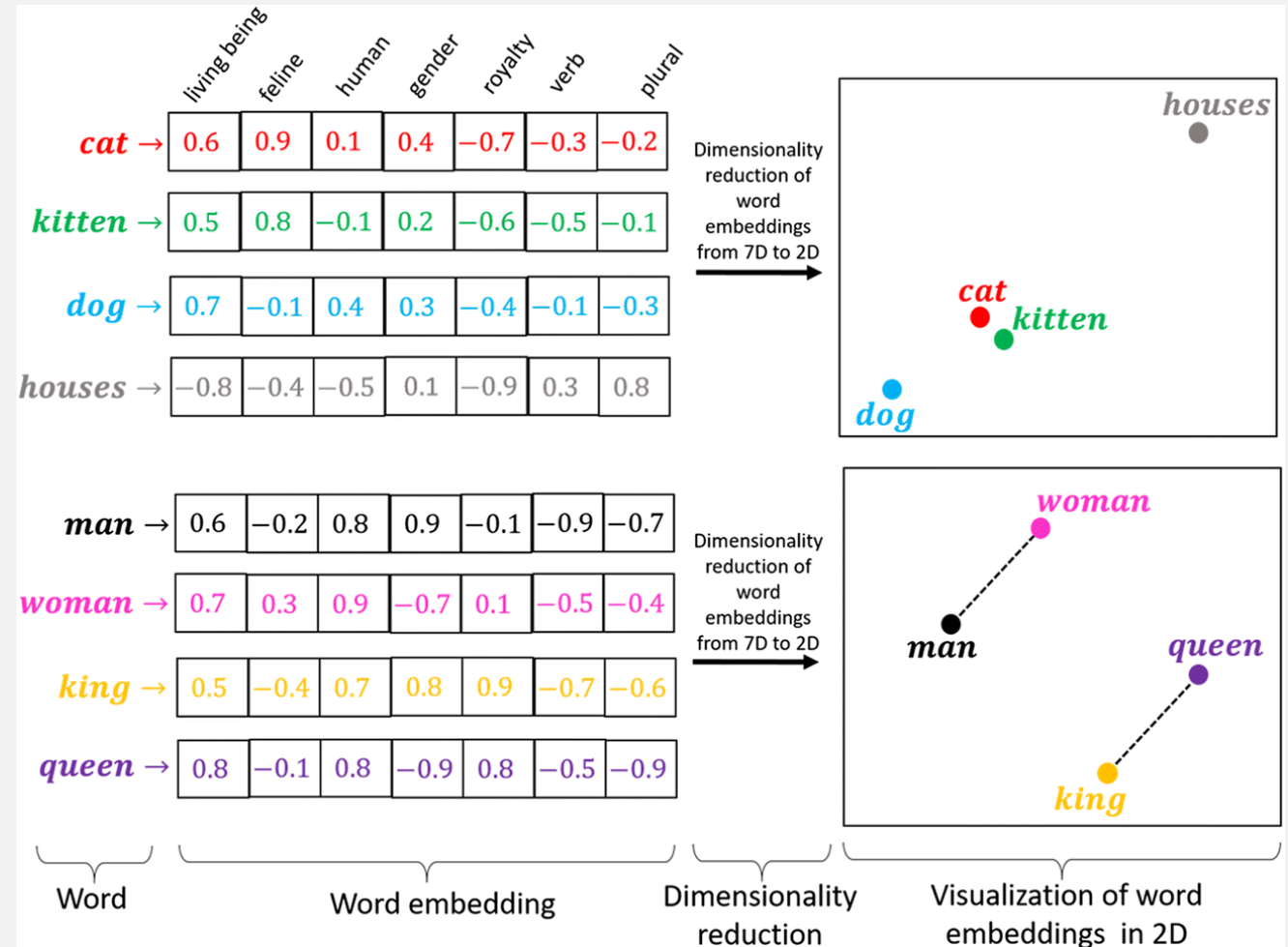
$\text{lizard} - \text{snake} = [0, 0, 4] \Rightarrow 4 \text{ additional legs}$

Text Vectorization - Embeddings

To represent words:

⇒ many more nuances and relationships that need to be captured.

⇒ requires a higher-dimensional space to represent these relationships effectively.



Text Vectorization - Word2Vec/ Doc2Vec

- The word2vec algorithm uses a neural network model to learn word associations from a large corpus of text.
- Word2vec can:
 - Detect synonymous words
 - Suggest additional words for a partial sentence
- Word2Vec represents each word with a vector. The closer 2 vectors are the more their words are **semantically close**

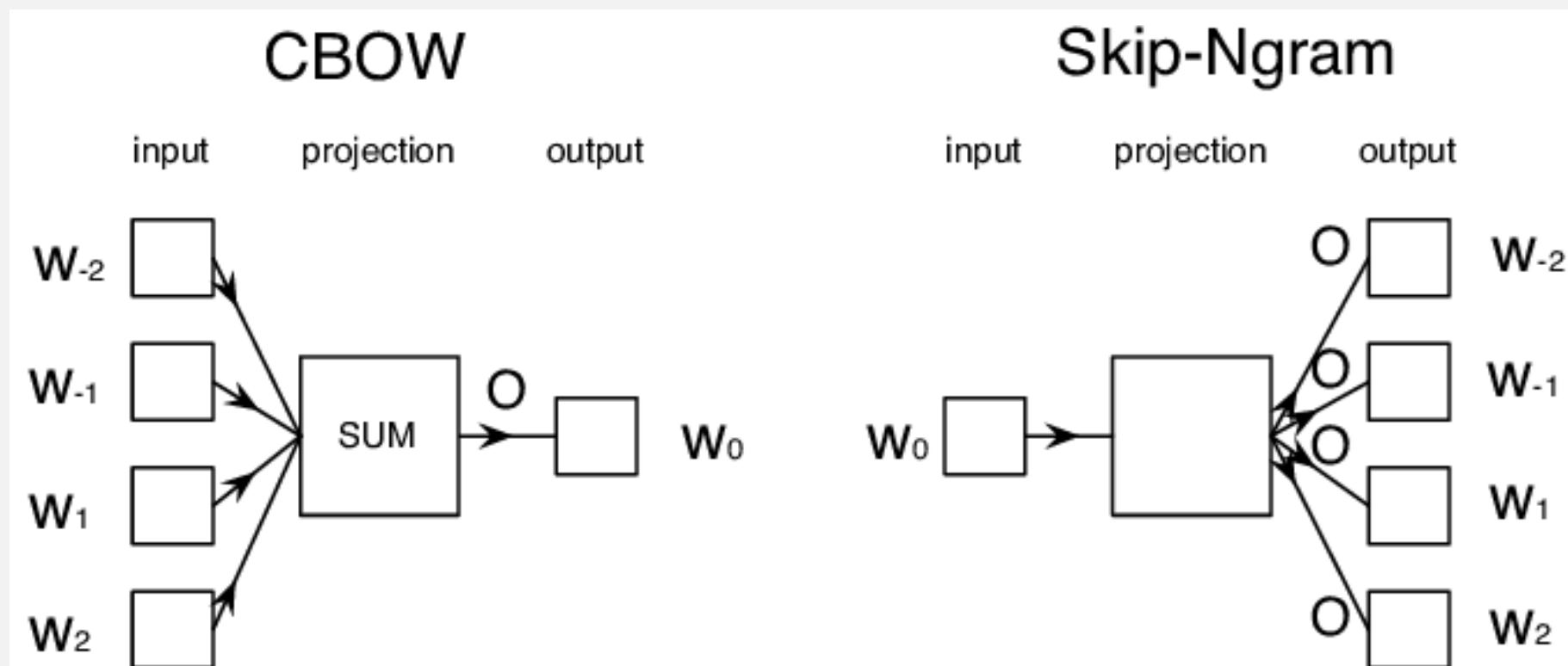
Text Vectorization - Word2Vec/ Doc2Vec

Representation of words using Word2Vec can be done in two major methods:

- **Continuous Bag of Words (CBOW) Method:** predicts a target word from surrounding context words.
- **Skip-Gram Method:** predicts context words from a given word.

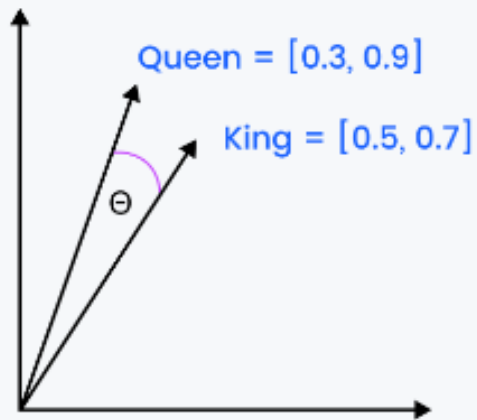
Example: Using Word2Vec, the nearest neighbors of 'Paris' might include 'France' and 'Eiffel Tower'.

Text Vectorization - Word2Vec/ Doc2Vec



Text Vectorization – Cosine Similarity

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$



$$\begin{aligned} \text{Cos}(\text{Queen}, \text{King}) &= \frac{(0.3 \cdot 0.5) + (0.9 \cdot 0.7)}{\sqrt{0.3^2 + 0.9^2} \cdot \sqrt{0.5^2 + 0.7^2}} \\ &= \frac{0.15 + 0.63}{\sqrt{0.9^2} \cdot \sqrt{0.74}} \\ &= \frac{0.78}{\sqrt{0.666}} \\ &= 0.03 \end{aligned}$$

Cos Similarity:

- = 1 indicates that the vectors are identical
- = 0 indicates that the vectors are orthogonal (no similarity)
- = -1 indicates that the vectors are diametrically opposed

Text Vectorization - Doc2Vec

- Extends Word2Vec to learn vector representations of entire documents rather than individual words.
- Uses paragraph vectors to capture document semantics.
- **Applications:**
 - Document clustering: Automatically grouping similar news articles together.
 - Information retrieval: Improving search engines by ranking relevant documents.
- **Example:** Using Doc2Vec, similar articles on "machine learning" can be grouped together even if they don't share exact word matches.

Text Vectorization - Word2Vec Exercise

- From gensim library use Word2vec and the word2vec google news 300 model to:
 - Get sample embeddings between two words
 - Get the similarity between those two words

Text Vectorization - GloVe

- GloVe (Global Vector) is a model used for the representation of words in the form of vectors
- Matrix factorization-based approach vs. prediction-based models like Word2Vec.
- It uses an unsupervised model to create the embeddings.
- The glove model uses the matrix factorization technique for word embedding on the word-context matrix

Text Vectorization - GloVe

Co-occurrence matrix

The fast cat wears no hat

The cat in th4 hat ran fast

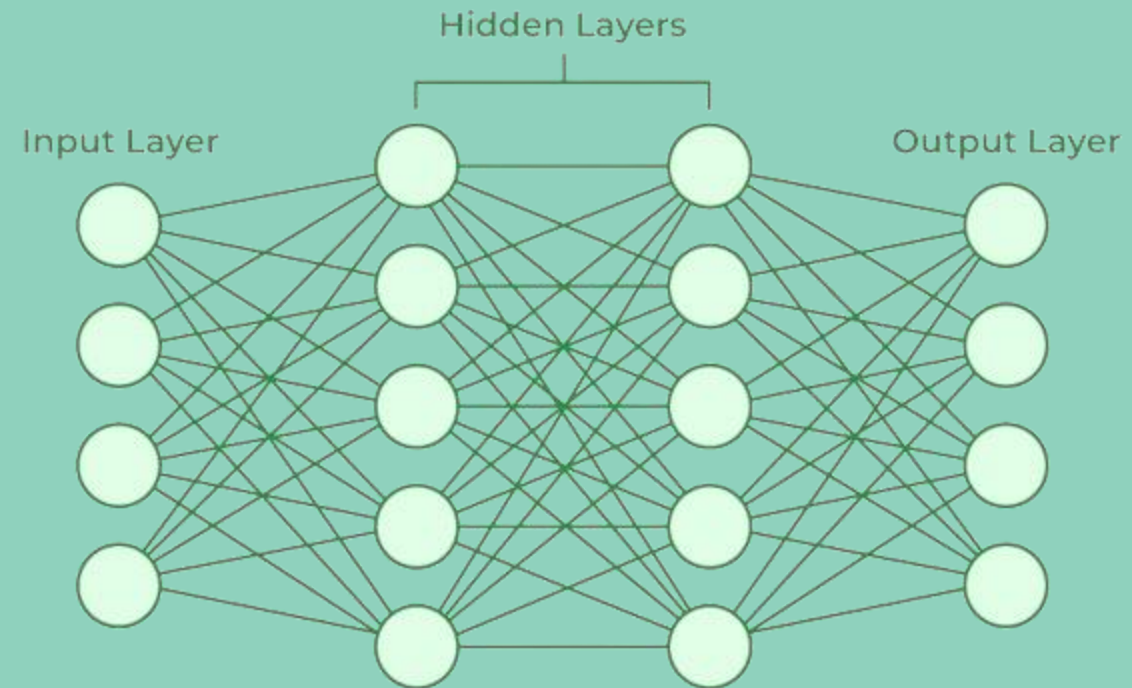
	cat	fast	hat	in	no	ran	the	wears
cat	0							
fast	2	0						
hat	2	2	0					
in	1	1	1	0				
no	1	1	1	0	0			
ran	1	1	1	1	0	0		
the	3	3	3	2	1	2	1	
wears	1	1	1	1	1	0	1	0

Text Vectorization - GloVe Exercise

- Use GloVe through Spacy to:
 - Get sample embeddings between two words
 - Get the similarity between those two words

Intro to Neural Networks

1. Definition
2. Component
 - 2.1. Neurons
 - 2.2. Layers
 - 2.3. Connections
3. Functionality
 - 3.1. Structure
 - 3.2. Activation Functions
 - 3.3. Forward & Backward Propagation



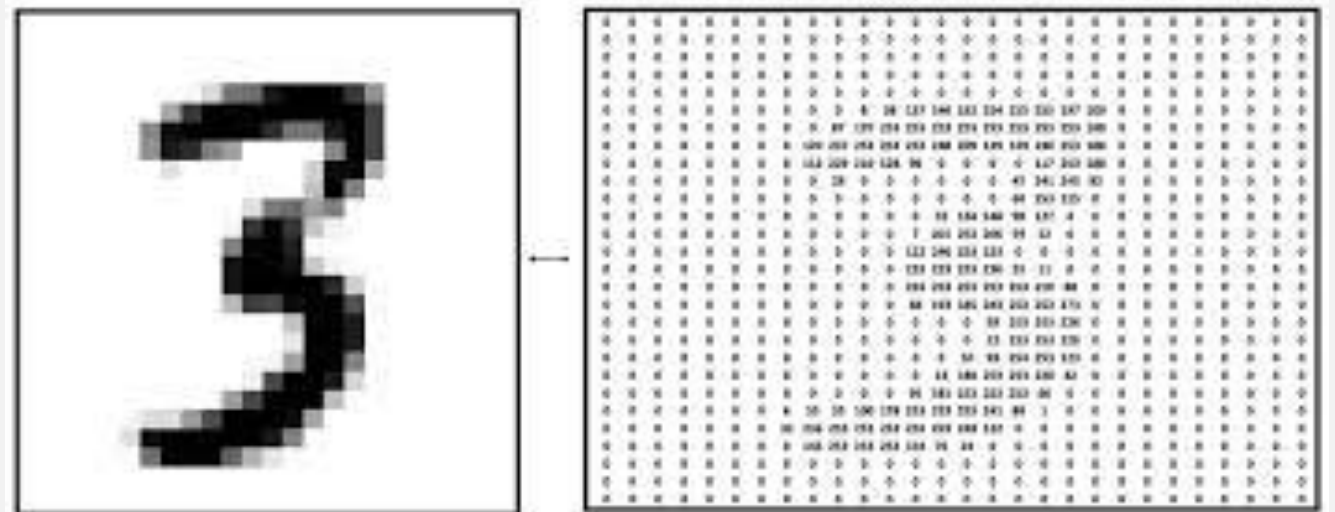
Neural Network - Definition

A neural network is a computational model inspired by the way biological neural networks in the human brain process information.

It consists of layers of interconnected nodes, or neurons, that work together to recognize patterns, make decisions, and learn from data.

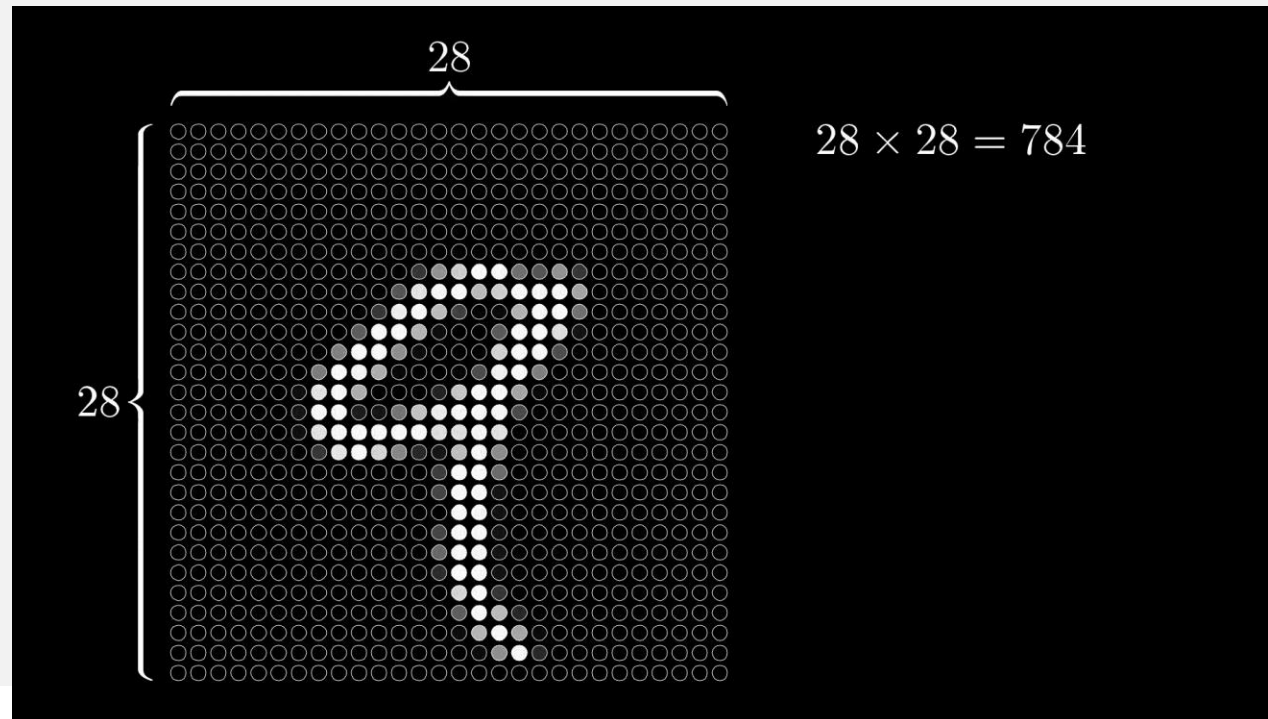
Think about how our brains recognize that this image represents the number three.

The patterns that make up the number three, are what we detect in our own neural network (our brain)



Neural Network - Definition

- To better understand neural networks, we'll be tackling the MNIST dataset, our input is basically a 28x28 grey scale image
- Our neural network will look like this:



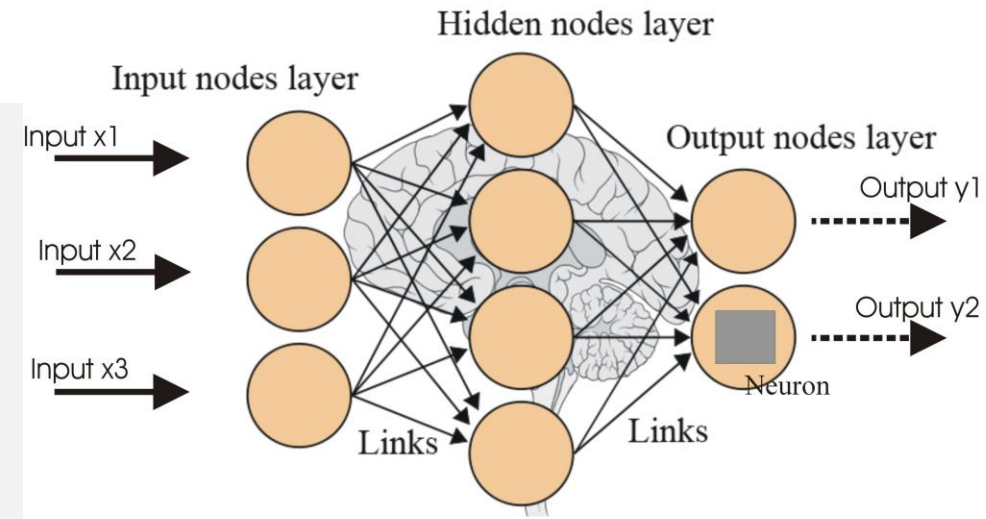
NN Components – Input Layer

Input Layer:

The first layer of the neural network.

It receives the initial data (features) and passes it to the next layer.

The number of neurons in this layer corresponds to the number of features in the input data.

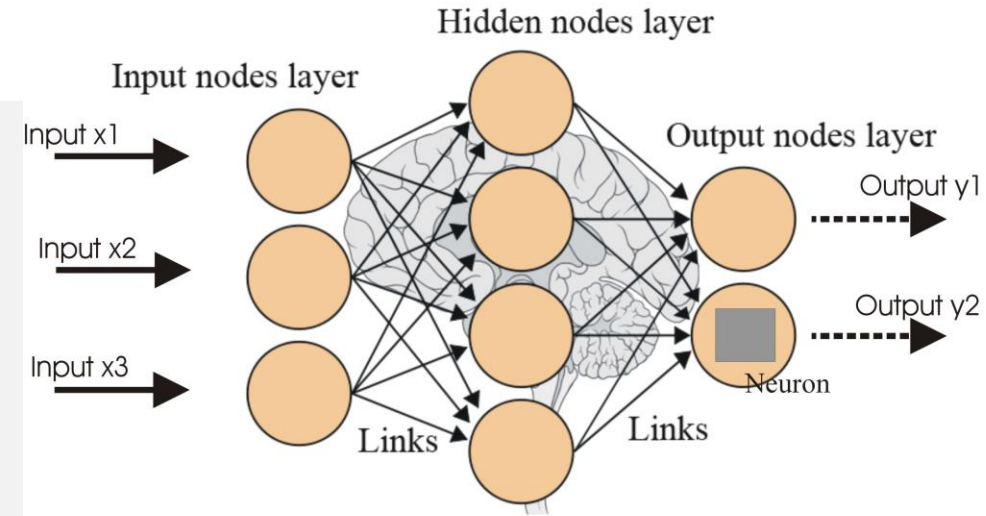


NN Components – Hidden Layers

Hidden Layers:

Layers between the input and output layers.

- They perform complex computations on the input data by applying weights and biases.
- Each neuron in a hidden layer receives input from all neurons in the previous layer, applies an activation function, and sends the output to the next layer.

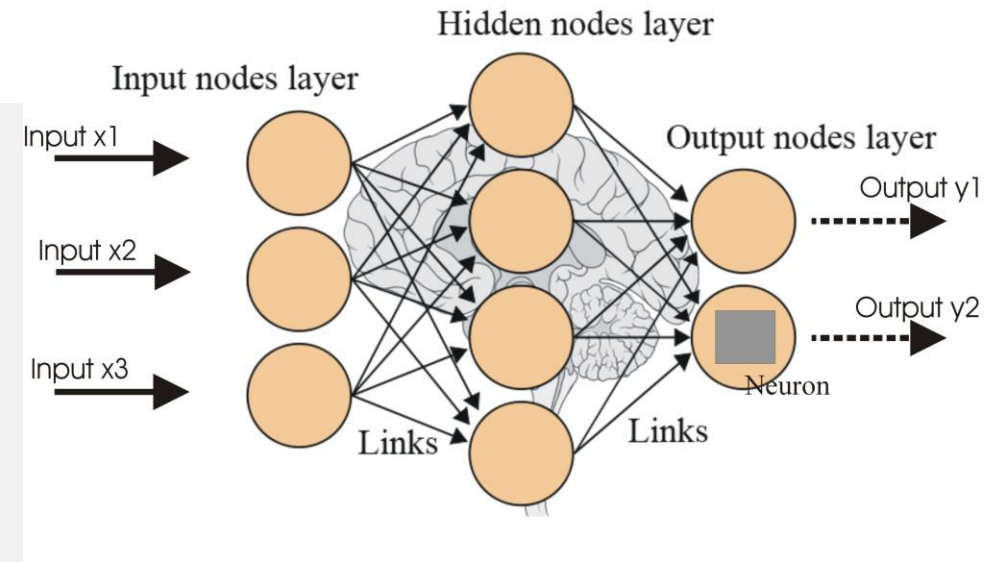


NN Components – Output Layer

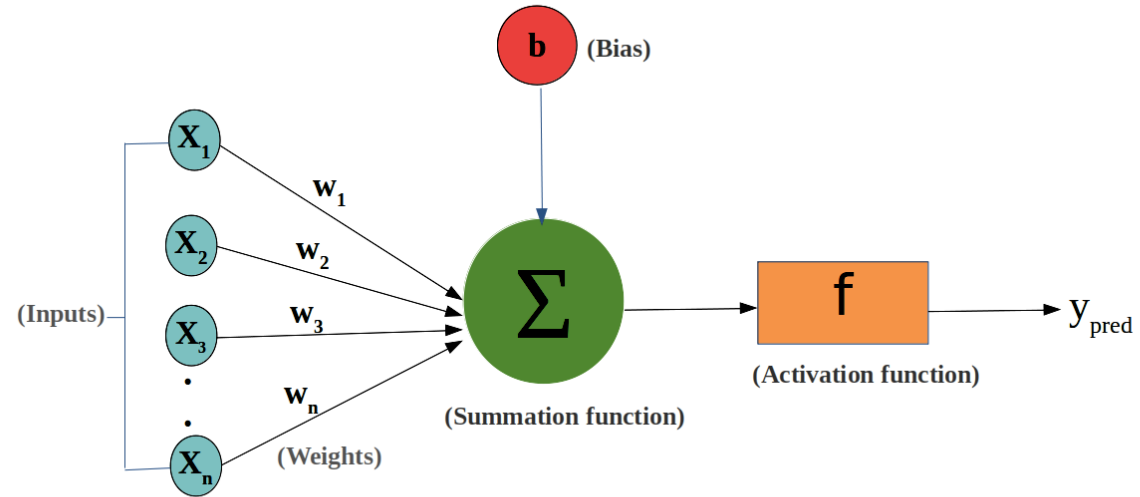
Output Layer:

It produces the output of the network, such as class probabilities in classification tasks or continuous values in regression tasks.

The number of neurons in this layer depends on the desired output (e.g., number of classes for classification).



NN Components – Neuron



Neurons:

Basic units of computation in the network.

Each neuron receives inputs, applies a weighted sum and a bias, and then passes the result through an activation function.

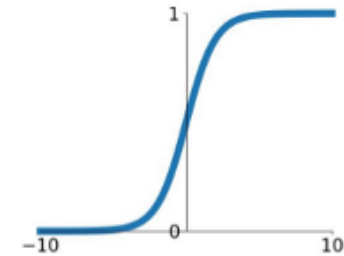
NN Components – Activation Function

Functions applied to the output of each neuron.

They introduce **non-linearity**, allowing the network to learn complex patterns.

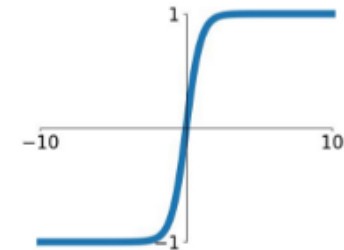
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



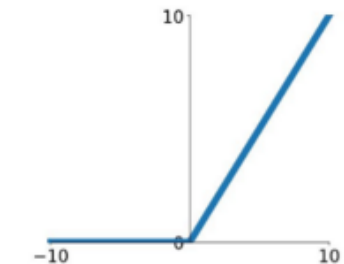
tanh

$$\tanh(x)$$



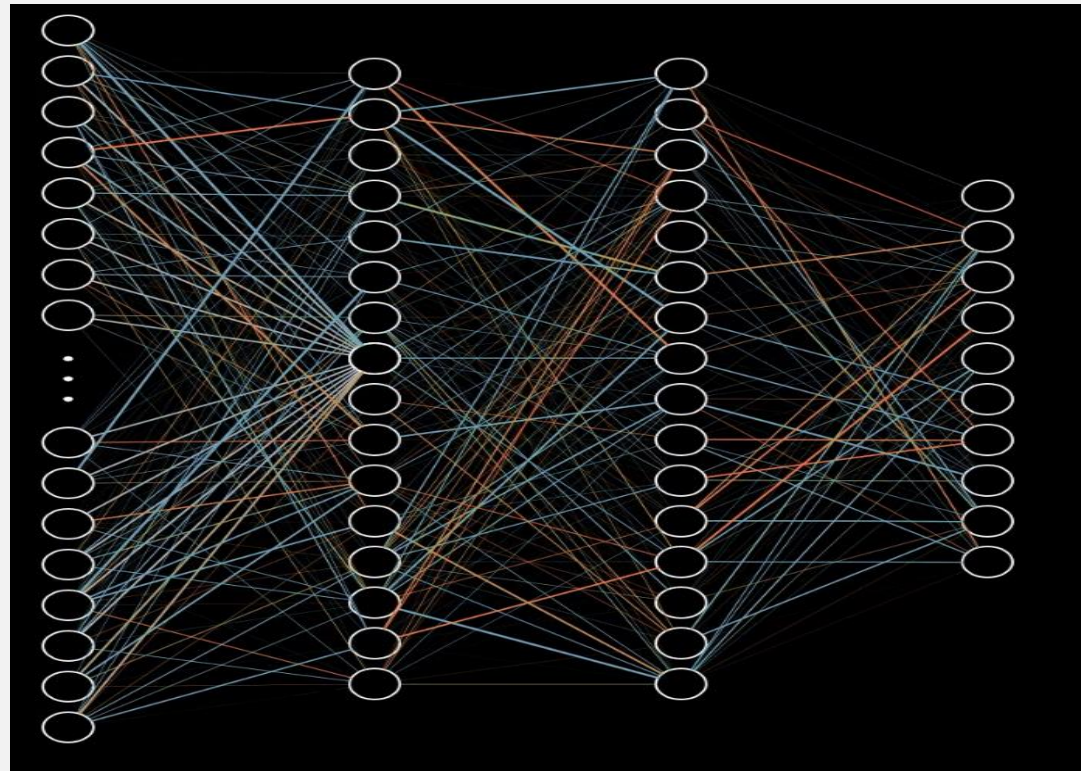
ReLU

$$\max(0, x)$$



Neural Network - Structure

So far, our network is just a connection that will give as outputs the probabilities for each class. The class with the highest probability will correspond the network's prediction result... Is that it?



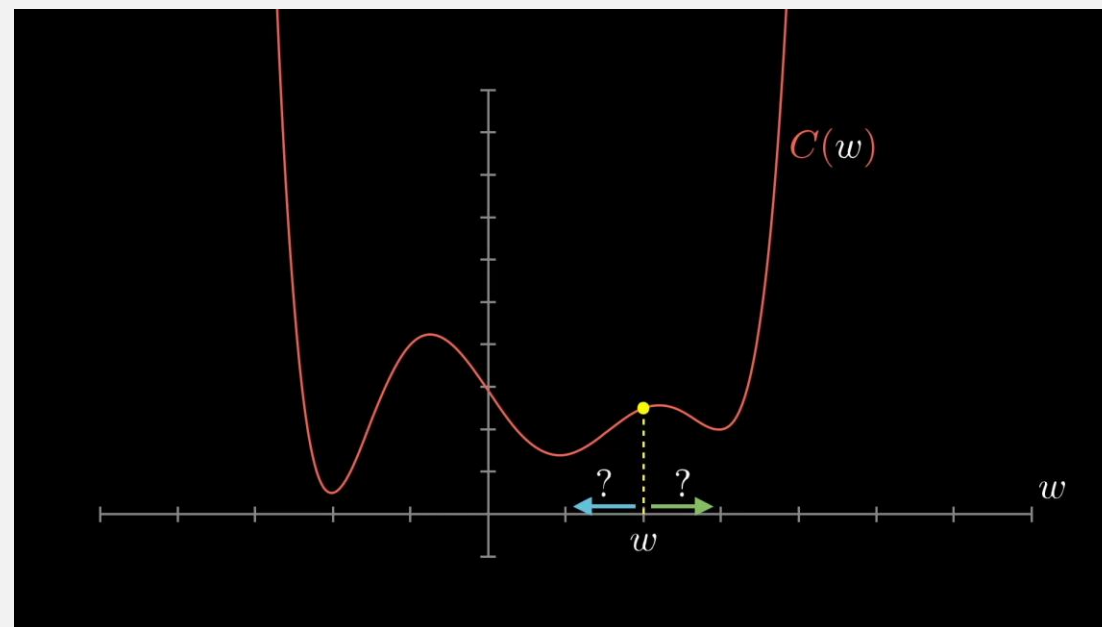
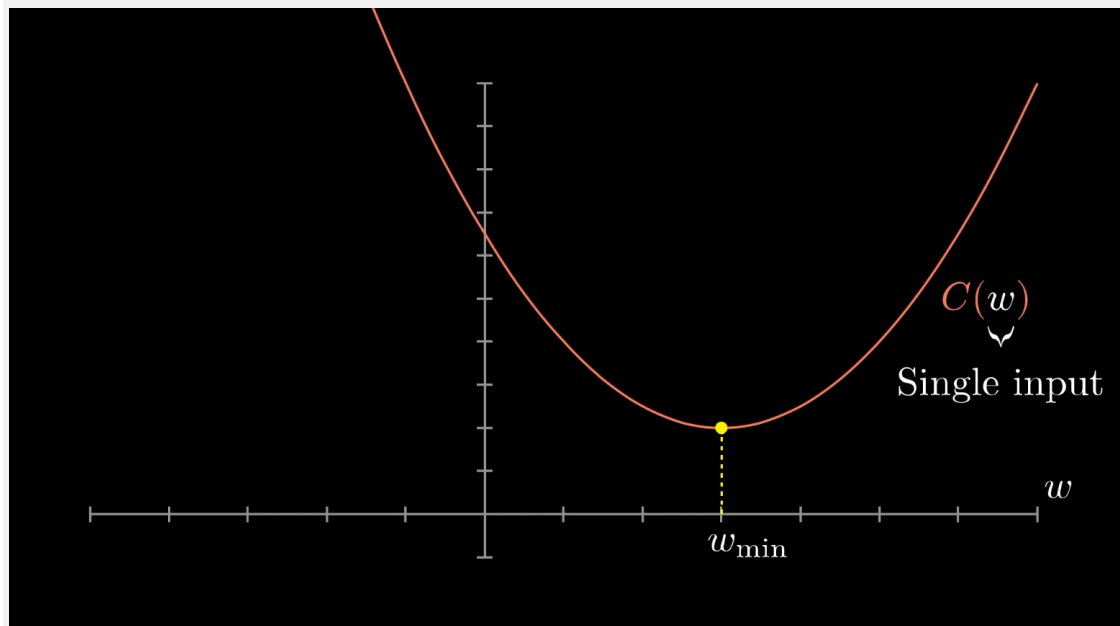
Neural Network - Learning

For learning to happen, we need to tell the network where it went wrong by using a **Loss Function**:

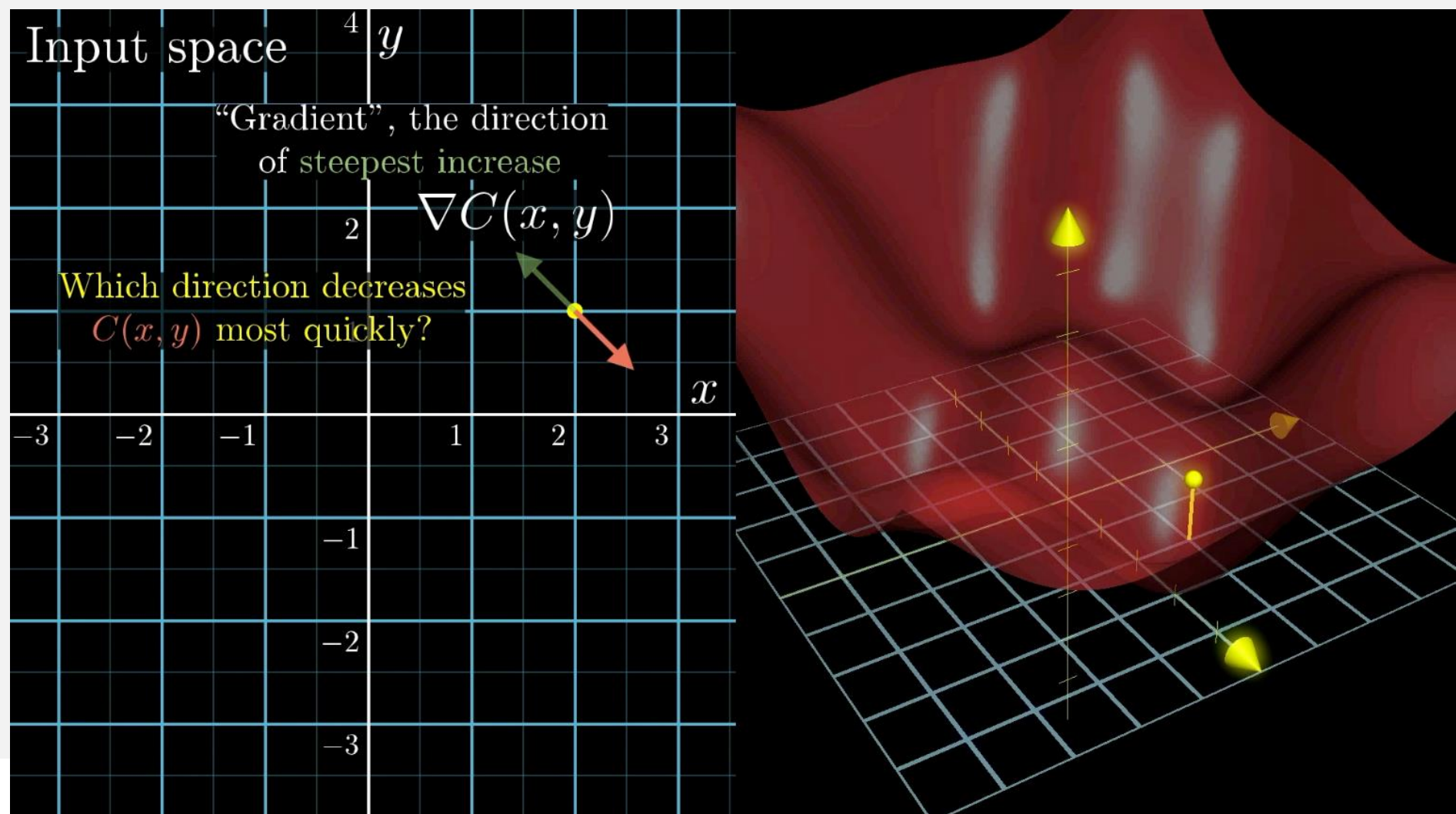
- It measures the error between the **predicted** output and the **actual** target.
- It guides the optimization process by providing a metric to **minimize** during training.

These functions may appear in many forms, such as the **MSE (Mean Squared Error)** for regression, or **categorical cross entropy** for classification problem

Neural Network - Learning

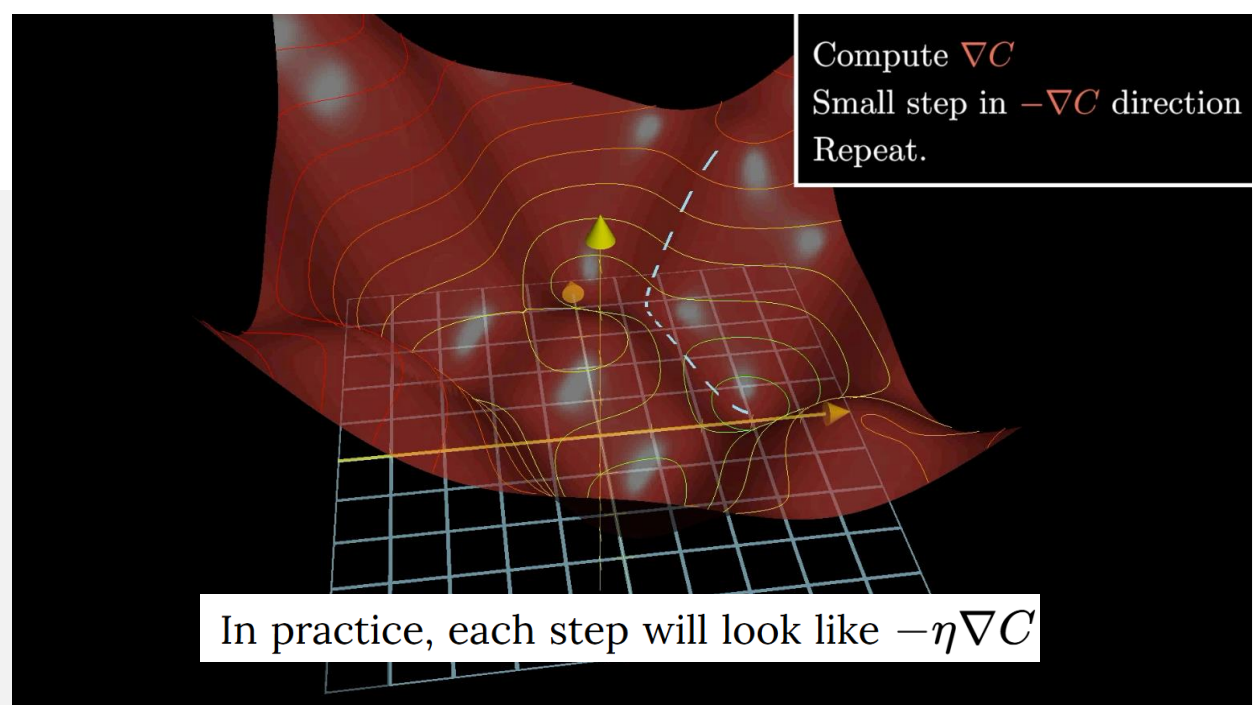


Neural Network - Learning



Neural Network - Learning

Now that we know what our cost looks like, we simply need to take small steps in order to change our weights and biases to decrease the cost



13,002 weights and biases

$$\vec{\mathbf{W}} = \begin{bmatrix} 2.25 \\ -1.57 \\ 1.98 \\ \vdots \\ -1.16 \\ 3.82 \\ 1.21 \end{bmatrix}$$

How to nudge all weights and biases

$$-\nabla C(\vec{\mathbf{W}}) = \begin{bmatrix} 0.18 \\ 0.45 \\ -0.51 \\ \vdots \\ 0.40 \\ -0.32 \\ 0.82 \end{bmatrix}$$

Neural Network - Learning

Optimizer:

An algorithm used to update the weights and biases during training.

It minimizes the loss function by adjusting the network parameters.

Popular optimizers include Stochastic Gradient Descent (SGD), Adam, etc

Neural Network – Training Concepts

- **Batch Size:** The number of training samples processed before the model's internal parameters (weights and biases) are updated.
Small Batch Size: More frequent updates, leading to faster convergence but noisier updates.
Large Batch Size: Smoother updates, which may be slower but more stable.
- **Epoch:** One complete pass through the entire training dataset.
Risk of overfitting: Too many epochs can lead to overfitting, where the model performs well on training data but poorly on unseen data. Early stopping can help prevent this.
- **Learning rate:** A hyperparameter that determines the step size at each iteration while moving toward a minimum of the loss function.
High Learning Rate: Faster learning but risks overshooting the minimum, leading to instability.
Low Learning Rate: Slower learning but more stable convergence, may get stuck in local minima.

Neural Network – Training Concepts

1. Initialization: The network's weights and biases are initialized, often randomly.
2. Forward Pass: The input data is passed through the network to get predictions.
3. Loss Calculation: The loss function computes the difference between the predictions and the true values.
4. Backward Pass: Backpropagation calculates the gradients of the loss function with respect to the network's weights.
5. Weight Update: Optimizers (like SGD, Adam) use these gradients to update the weights and biases in the direction that minimizes the loss.
6. Steps 2-5 are repeated for each batch of data within an epoch.
7. After one epoch, the entire dataset has been used for training, and the process may be repeated for multiple epochs until the model converges or early stopping is triggered.

Team Presentations

THANK YOU

ADDRESS

Mkalles, main road, bld. 757,
Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA

